Matrix Calculator

Made By 함중욱

INDEX

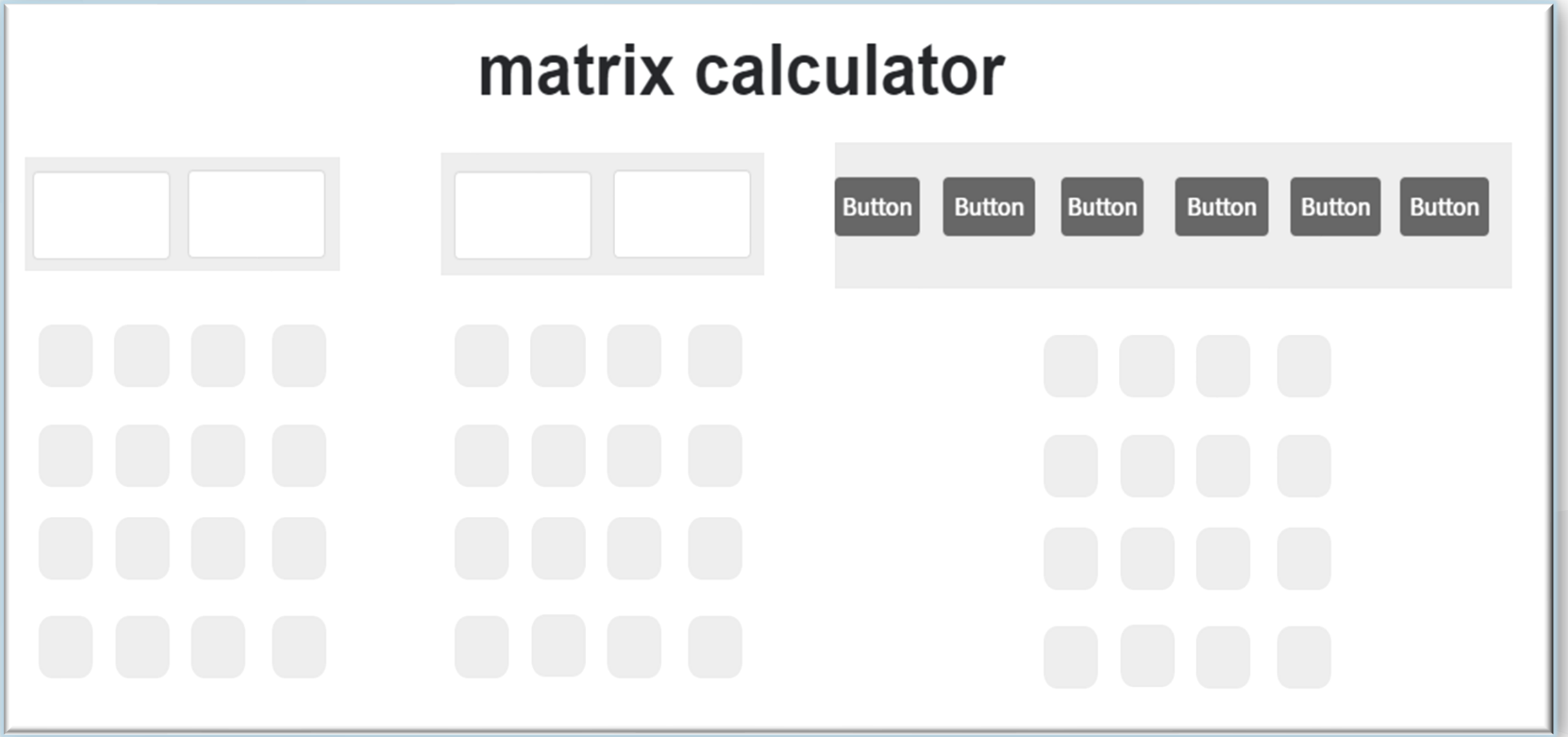
I 스케치 p.2

II 행렬 계산기 소개 p.3~5

III 코딩 p.6~17

IV 후기 p.18

스케치


matrix calculator
Button
Button
Button
Button
Button
Button

행렬 계산기 소개

전체화면

Matrix Calculator

9 9

74	38	45	24	27	27	61	58	79
48	86	97	73	64	55	94	48	99
50	60	31	42	29	1	18	16	70
81	17	64	9	94	61	26	61	45
97	76	71	39	55	47	47	58	31
67	98	7	31	18	20	58	48	40
76	7	80	90	45	4	56	35	95
64	49	84	34	10	32	79	60	70
4	84	54	13	76	15	18	14	78

9 9

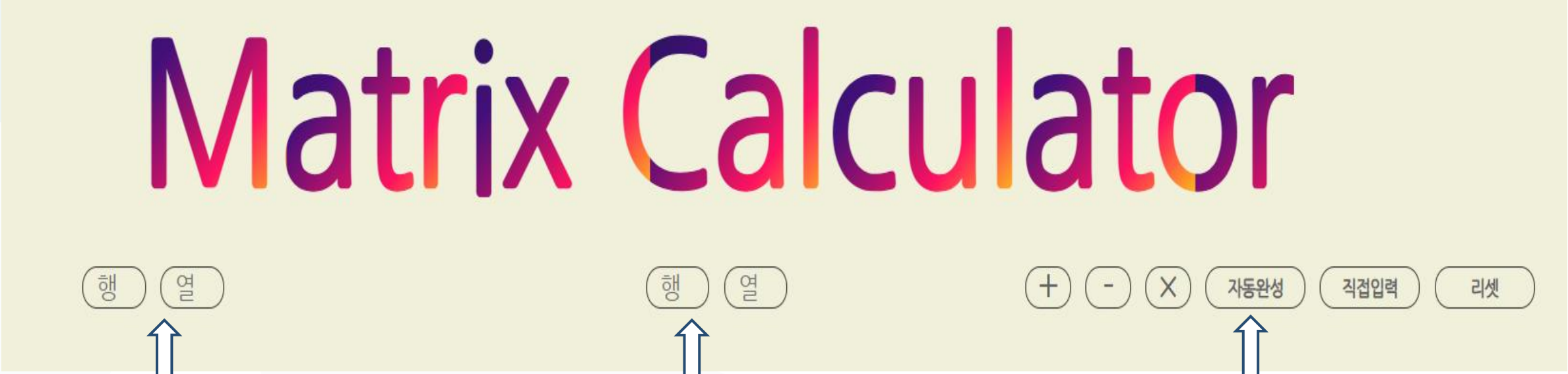
92	81	55	1	63	46	58	90	37
3	16	27	22	22	78	56	95	69
40	6	94	44	55	93	78	34	48
85	69	78	95	94	87	21	20	23
80	51	86	74	42	22	60	63	35
21	54	46	84	15	75	90	27	34
49	16	53	86	39	27	5	82	60
59	92	1	2	60	14	47	54	3
66	39	23	65	81	0	88	5	58

+ - × 자동완성 직접입력 리셋

166	119	100	25	90	73	119	148	116
51	102	124	95	86	133	150	143	168
90	66	125	86	84	94	96	50	118
166	86	142	104	188	148	47	81	68
177	127	157	113	97	69	107	121	66
88	152	53	115	33	95	148	75	74
125	23	133	176	84	31	61	117	155
123	141	85	36	70	46	126	114	73
70	123	77	78	157	15	106	19	136

행렬 계산기 소개

이용방법



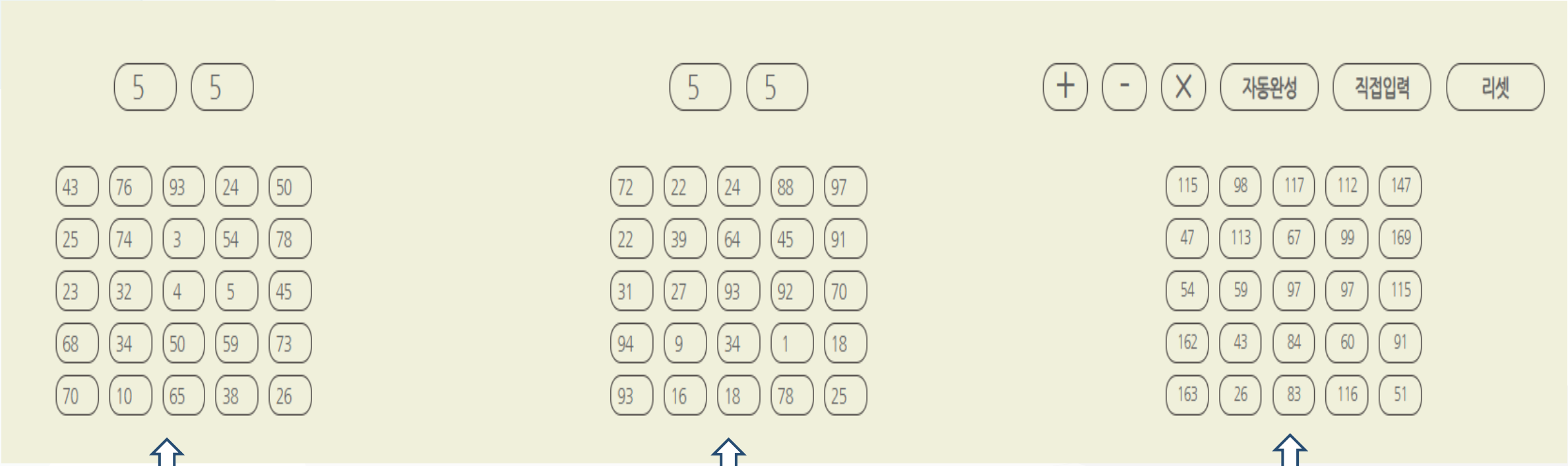
1. 행과 열의 개수를 입력
   1~9의 숫자 중 입력합니다.

2. 행과 열의 개수를 입력
   1~9의 숫자 중 입력합니다.

3. 자동완성: 1,2의 입력된 행과 열의 값이 1~99까지 랜덤으로 입력 됩니다.
   직접입력: 입력된 행과 열의 값을 직접 입력합니다.
   리셋: 1,2의 행과 열의 값을 초기화합니다.
   더하기: 1,2의 행과 열의 값을 더합니다.
   빼기: 1,2의 행과 열의 값을 뺍니다.
   곱하기: 1,2의 행과 열의 값을 곱합니다.

행렬 계산기 소개

이용방법



4. 행과 열의 값이 생성됩니다.

5. 행과 열의 값이 생성됩니다.

6. 좌 우의 행과 열의 값의 결과 값을 도출합니다.

코딩

Html

```
section(class='mc_input_array_box')
  section(class='inputArray')
    input(id='xinputArray' class='xinputArray' type='number' title='xinputArray' min='1' max='9' value='' placeholder='행')
    input(id='yinputArray' class='yinputArray' type='number' title='yinputArray' min='1' max='9' value='' placeholder='열')

  section(class='inputArray2')
    input(id='xinputArray2' class='xinputArray2' type='number' title='xinputArray2' min='1' max='9' value='' placeholder='행')
    input(id='yinputArray2' class='yinputArray2' type='number' title='yinputArray2' min='1' max='9' value='' placeholder='열')

  section(class='inputArray3')
    i(id='button_box_plus' class='button_box_plus' class='xi-plus xi-5x')
    i(id='button_box_minus' class='button_box_minus' class='xi-minus xi-5x')
    i(id='button_box_multi' class='button_box_multi' class='xi-close xi-5x')
    input(id='button_box_random' class='button_box_random' type='button' title='button_box_random' value='자동완성')
    input(id='button_box_print' class='button_box_print' type='button' title='button_box_print' value='직접입력')
    input(id='button_box_reset' class='button_box_reset' type='button' title='button_box_reset' value='리셋')
```

좌측 행과 열의 값을 입력받기 위한 영역입니다.

우측 행과 열의 값을 입력받기 위한 영역입니다.

더하기, 빼기, 곱하기, 자동완성, 직접입력, 리셋 버튼 영역입니다.

코딩

Html

```
42
43      section(class='mc_small_box')
44        section(class='xprintArray')
45          section(id='xprint_Array' class='xprint_Array')
46
47        section(class='yprintArray')
48          section(id='yprint_Array' class='yprint_Array')
49
50        section(class='printArray')
51          section(id='print_Array' class='print_Array')
52
53        section(id='alert_event_box' class='alert_event_box')
54          p(id='alert_box' class='alert_box')
55
```

좌우의 입력된 행렬의 값이 그려지는 영역입니다.

좌우의 입력된 행렬의 값을 연산 후 그려지는 영역입니다.

경고 문구를 출력하는 영역입니다.

코딩

Script

```
inputArray (idName) {
  this.valueArray = [];
  this.xprint_Array = '';
  let i = 0;
  while (i < this.xArray) {
    this.valueArray.push([]);
    let j = 0;
    while (j < this.yArray) {
      this.valueArray[i][j] = 0;
      this.xprint_Array += '<input id="' + idName + i + "_" + j + '" class="miniBox" type="number" value="0">';
      j++;
    }
    i++;
  }
  return this.xprint_Array;
}
```

직접입력 버튼 클릭 시 그려지는 행렬의 메소드입니다.

코딩

Script

```
randomArray (idName) {
  this.valueArray = [];
  this.xprint_Array = '';
  let i = 0;
  while (i < this.xArray) {
    this.valueArray.push([]);
    let j = 0;
    while (j < this.yArray) {
      this.valueArray[i][j] = Math.floor(Math.random() * 100);
      this.xprint_Array += '<input id="' + idName + i + "_" + j + '" class="miniBox" type="number" value="' + this.
      valueArray[i][j] + '">';
      j++;
    }
    i++;
  }
  return this.xprint_Array;
}
```

자동완성 버튼 클릭 시 그려지는 배열 메소드

코딩

Script

```
plusArray () {
  this.calArray = '';
  let i = 0;
  while (i < this.xArray) {
    let j = 0;
    while (j < this.yArray) {
      const resVal = (parseInt($("#inputA" + i + "_" + j).val()) + parseInt($("#inputB" + i + "_" + j).val()));
      this.calArray += '<input class="miniBox" type="text" value="' + resVal + '" readonly>';
      j++;
    }
    i++;
  }
}
```

두개의 배열값을 더한
결과를 보여주는 메소드

코딩

Script

```
minusArray () {
  this.calArray = '';
  let i = 0;
  while (i < this.xArray) {
    let j = 0;
    while (j < this.yArray) {
      const resVal = (parseInt($("#inputA" + i + "_" + j).val()) - parseInt($("#inputB" + i + "_" + j).val()));
      this.calArray += '<input class="miniBox" type="text" value="' + resVal + '" readonly>';
      j++;
    }
    i++;
  }
}
}
```

두개의 배열값을 뺀
결과를 보여주는 메소드

코딩

Script

```
const alertEvent = (selectors, body) => {
  $(".alert_event_box").css("display", "block");
  $("#alert_box").html(body);
  if (selectors) {
    $(selectors).val("");
  }
  $(".alert_event_box").off("click").on("click", function () {
    $(this).css("display", "none");
  });
}
```

배열의 입력 없이 버튼 클릭시 알럿 띄우는 메소드

코딩

Script

```
$("input").on("keyup", function () {
  const checkIn = /^[-0]|[\.]/g;
  if (checkIn.test($(this).val())) {
    alertEvent(this, "1~9까지 입력해 주세요.");
    $("#xprint_Array, #yprint_Array, #print_Array").empty();
  }
});
```

좌우의 배열 숫자를 9이상 입력시 알럿을 띄우는 메소드

코딩

Script

```
const multiArray = () => {
  let newArray = '';
  for (let i = 0; i < xinputValue; i++) {
    for (let j = 0; j < yinputValue2; j++) {
      let totalValue = 0;
      for (let k = 0; k < yinputValue; k++) {
        totalValue += parseInt($('#inputA' + i + "_" + k).val()) * parseInt($('#inputB' + k + "_" + j).val());
      }
      newArray += '<input class="miniBox" type="text" value="' + totalValue + '" readonly>';
    }
  }
  return newArray;
}
```

곱하기 버튼 클릭 시
입력된 좌우 행렬의 값을
빼기 연산하는 영역입니다.

코딩

Css(Sass)

```
@charset "UTF-8";

* {
  margin: 0px;
  padding: 0px;
  font: {
    size: 10px;
    family: 'Nanum Gothic', sans-serif;
  }
}

@import url('https://fonts.googleapis.com/css2?family=Nanum+Gothic&display=swap');

html, body {
  width: 100vw;
  height: 100vh;
  background-color: #F0F0DB;
}

```

Sass를 이용하여 빠르고 간단하게 작성
전체 화면 크기 및 백그라운드 색상 지정

코딩

Css(Sass)

```sass
@mixin mc_main_box {
  width: 100vw;
  height: 100vh;
}

@mixin inputArray {
  width: 10vw;
  height: 10vh;
  display: flex;
  align-items: center;
}

@mixin xyinputArray {
  width: 5vw;
  height: 5vh;
  border: 0.1vw solid #393434;
  border-radius: 0.5vw;
  background-color: #F0F0DB;
  margin: auto 0.5vw;
}

```

```sass
@mixin buttonBox {
  width: 5vw;
  height: 5vh;
  border: 0.1vw solid #393434;
  border-radius: 0.5vw;
  background-color: #F0F0DB;
  margin: auto 0.5vw;
  color: rgb(100,100,100);
}
@mixin buttonBox2 {
  width: 5vw;
  height: 5vh;
  border: 0.1vw solid #393434;
  border-radius: 0.5vw;
  background-color: #F0F0DB;
  margin: auto 0.5vw;
  font: {
    size: 1.1vw;
    weight: bold;
  }
  color: rgb(100,100,100);
}
```

Mixin를 이용하여 빠르고 간단하게 작성
컨텐츠의 크기 폰트 크기 등을 지정

코딩

Css(Sass)

```
.mc_main_box .mc_mini_box .mc_header_box {
  width: 100vw;
  height: 10vh;
  display: flex;
  justify-content: center;
  margin-top: 5vh;
}
.mc_main_box .mc_mini_box .mc_header_box .mc_header_font {
  font-size: 5vw;
  font-weight: bold;
  background-image: linear-gradient(-225deg, #231557 0%, #44107a 29%, #ff1361 67%, #fff800 100%);
  background-size: 10vw auto;
  -webkit-background-clip: text;
  -webkit-text-fill-color: transparent;
  -webkit-animation: textclip 20s linear infinite;
          animation: textclip 20s linear infinite;
}
```

헤더 폰트의 색상 컬러 그라데이션 애니메이션 지정

후기

프로젝트를 처음 접하면서 어떤 식으로 UX/UI를 구성해야 하는지 어떤 식으로 로직을 구성해야 하는지 감이 오지 않아 힘들었고 UX/UI를 먼저 구성을 하고 많은 고민과 수정 등을 거쳐 부족하지만 완성을 하였다.

프로젝트를 진행하면서 다양한 방법의 로직을 생각해 보는 계기가 되었고 많이 미숙하지만 객체, 함수, 반복문 등의 사용에 대해 이해하는 시간이 된 것 같다

많은 시간을 들여서 프로젝트를 하였으나 아직 많은 부분이 부족하여 더 공부하여서 개선을 해야 함을 느꼈습니다.